



NLP UNIT 1 Tokeni - notess

Natural Language Processing (Swarnandhra College of Engineering and Technology)



Scan to open on Studocu

1. Tokenization

Tokenization is the process of breaking a text sequence into smaller meaningful units called *tokens*. These tokens may be **words**, **subwords**, **characters**, or **sentences**, depending on the application.

At the **word level**, tokenization splits text using delimiters such as spaces and punctuation. For example, the sentence “*NLP improves text understanding.*” is tokenized as ["NLP", "improves", "text", "understanding"]. Sentence-level tokenization segments paragraphs into sentences, while character-level tokenization splits text into individual characters. Modern NLP systems often use **subword tokenization** (such as Byte Pair Encoding or WordPiece) to effectively handle unknown words, rare terms, and morphologically rich languages.

Tokenization standardizes text input, enables vocabulary construction, and serves as the foundation for statistical and neural language models.

2. Detecting Spelling Errors

Spelling error detection identifies words that do not conform to valid dictionary entries or learned language patterns. Spelling errors are broadly categorized into:

- **Non-word errors**: Invalid words not found in a dictionary (e.g., “*recieve*”).
- **Real-word errors**: Valid words used incorrectly in context (e.g., “*their*” instead of “*there*”).

Traditional detection methods rely on **dictionary lookup**, where tokens not present in a lexicon are flagged as errors. However, this approach cannot detect real-word errors. More advanced methods use **n-gram language models**, **part-of-speech constraints**, or **context-aware neural models** to identify improbable word sequences.

3. Correcting Spelling Errors

Spelling correction aims to replace erroneous tokens with the most probable intended word. The classical approach is based on **Minimum Edit Distance**, which measures how many operations (insertion, deletion, substitution, transposition) are required to transform a misspelled word into a valid one. For example, correcting “*speling*” to “*spelling*” requires a single insertion.

Modern correction systems improve accuracy by combining:

- **Candidate generation** using edit distance
- **Candidate ranking** using language models
- **Contextual probability estimation** using neural networks (e.g., transformers)

In context-sensitive correction, the system selects the correction that best fits the surrounding words, not just the closest spelling.

4. Importance in NLP Applications

Tokenization and spelling correction significantly improve **text normalization**, reduce vocabulary sparsity, and enhance semantic understanding. In applications such as search engines, chatbots, OCR systems, and speech-to-text pipelines, accurate spelling correction ensures better user interaction and reliable output.

In summary, tokenization structures raw text into analyzable units, while spelling detection and correction improve linguistic accuracy. Together, they form a critical preprocessing backbone for robust and high-performing NLP systems.

1. Tokenization (with Examples)

Tokenization is the process of breaking text into smaller units called *tokens*. These tokens are the basic input units for NLP models.

Example Sentence

“Natural Language Processing is powerful.”

(a) Word Tokenization

Splits text into individual words.

Output:

["Natural", "Language", "Processing", "is", "powerful"]

(b) Sentence Tokenization

Splits a paragraph into sentences.

Text:

“NLP is useful. It is widely used in AI.”

Output:

["NLP is useful.", "It is widely used in AI."]

(c) Character Tokenization

Splits text into characters.

Word: "NLP"

Output:

["N", "L", "P"]

(d) Subword Tokenization

Used in modern NLP models to handle unknown or rare words.

Word: "unhappiness"

Output:

["un", "happi", "ness"]

Why tokenization matters:

It converts raw text into a structured form that machines can analyze efficiently.

2. Detecting Spelling Errors (with Examples)

Spelling error detection identifies incorrect words in text.

Types of Spelling Errors

(a) Non-Word Errors

These words do **not exist** in the dictionary.

Example Sentence:

"I recieved the mesage."

Detected Errors:

recieved, mesage

(b) Real-Word Errors

These words exist in the dictionary but are **incorrect in context**.

Example Sentence:

"She went to there house."

All words are valid, but **"there"** should be **"their"**.

3. Correcting Spelling Errors (with Examples)

(a) Dictionary-Based Correction

Replace misspelled words with the closest dictionary word.

Input:

recieve

Correction:

receive

(b) Minimum Edit Distance Method

This method calculates the **minimum number of edits** needed to convert one word into another.

Allowed Operations:

- Insertion
- Deletion
- Substitution
- Transposition

Example

Correct "speling" → "spelling"

Edits Required:

- Insert 'l'

Edit Distance = 1

The word with the **minimum edit distance** is chosen as the correction.

(c) Context-Based Correction

Uses surrounding words to choose the correct spelling.

Sentence:

"He lost his write hand."

Possible Corrections for "write":

- right
- write

Corrected Sentence:

"He lost his right hand."

Context helps choose the semantically correct word.

4. Complete NLP Example (End-to-End)

Input Sentence

"Ths is a smple sentnce."

Step 1: Tokenization

["Ths", "is", "a", "smple", "sentnce"]

Step 2: Error Detection

Ths, smple, sentnce

Step 3: Error Correction

This, simple, sentence

Final Output

“This is a simple sentence.”

5. Where This Is Used (Real Applications)

- **Search engines** (query correction)
- **OCR systems** (scanned text correction)
- **Chatbots & virtual assistants**
- **Speech-to-text systems**
- **Text mining & sentiment analysis**

Summary

- **Tokenization** splits text into meaningful units.
- **Spelling detection** finds incorrect or unlikely words.
- **Spelling correction** replaces them using edit distance and context.
- Together, they significantly improve NLP accuracy and reliability.

Minimum Edit Distance (MED) is a fundamental concept in NLP used to measure how different two strings (words) are. It is defined as the **minimum number of editing operations** required to transform one word into another. MED is widely used in **spelling correction, OCR error correction, speech recognition, and information retrieval**.

1. Edit Operations in MED

To convert a source word XXX into a target word YYY, the following operations are allowed:

1. **Insertion (I)** – Insert a character
Example: bok → book (insert **o**)
2. **Deletion (D)** – Remove a character
Example: books → book (delete **s**)
3. **Substitution (S)** – Replace one character with another
Example: cat → cut (substitute **a** with **u**)

Each operation usually has a **cost of 1** (though weighted costs can also be used).

2. Why Minimum Edit Distance is Important

In spelling correction, a misspelled word is corrected by choosing the dictionary word with the **smallest edit distance**.

Example:

Misspelled word: recieve

Candidate corrections: receive, recipe, relieve

The word **receive** has the **minimum edit distance**, so it is selected.

3. Simple Example (Manual Calculation)

Example 1: Convert "cat" → "cut"

Operation	Result
-----------	--------

Substitute a → u	cut
--------------------------------	-----

Edit Distance = 1

Example 2: Convert "kitten" → "sitting"

Steps:

1. kitten → sitten (substitute **k** → **s**)
2. sitten → sittin (substitute **e** → **i**)
3. sittin → sitting (insert **g**)

Edit Distance = 3

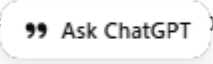
4. MED Using Dynamic Programming

MED is efficiently computed using Dynamic Programming (DP).

Let:

- $X = x_1x_2 \dots x_m$ (source word)
- $Y = y_1y_2 \dots y_n$ (target word)

We build a matrix $D(m+1, n+1)$, where:

- $D[i][j]$:  convert first i characters of X into first j characters of Y

Initialization

- $D[i][0] = i$ (delete all characters)
- $D[0][j] = j$ (insert all characters)

Recurrence Relation

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & \text{(Deletion)} \\ D[i][j-1] + 1 & \text{(Insertion)} \\ D[i-1][j-1] + cost & \text{(Substitution)} \end{cases}$$

where

$cost = 0$ if $x_i = y_j$, else 1

5. Worked Example with Table

Convert "intention" $\rightarrow$ "execution"

The DP table is filled row by row using the recurrence relation.

The final cell $D[m][n]$ gives the minimum edit distance.

✓ Minimum Edit Distance = 5

This means 5 edits are required to convert *intention* into *execution*.

6. MED in Spelling Correction (NLP Example)

Sentence:

"This is a smple text."

Tokenization:

CSS

 Copy code

```
["This", "is", "a", "smple", "text"]
```

Candidate Corrections for "smple":

Word	Edit Distance
simple	1
sample	2
smile	2

✓ Chosen Correction: simple

Minimum Edit Distance (MED): Step-by-Step DP Table Construction

Let us clearly construct the Dynamic Programming (DP) table step by step using a simple and exam-friendly example, exactly how it is expected in NLP / Compiler / AI exams.

Example

Convert the word:

Source word (X): cat

Target word (Y): cut

Allowed operations and cost:

- Insertion = 1
 - Deletion = 1
 - Substitution = 1 (0 if characters are same)
-

Step 1: Define the DP Table

Let

- $X = cat$ (length = 3)
- $Y = cut$ (length = 3)

Create a DP table D of size $(m+1) \times (n+1) = 4 \times 4$

Rows $\rightarrow$ source word (cat)

Columns $\rightarrow$ target word (cut)

Step 2: Initialize the Table

- First row: converting empty string to target → **insertions**
- First column: converting source to empty string → **deletions**

	ε	c	u	t
ε	0	1	2	3
c	1			
a	2			
t	3			

Step 3: Fill the Table Cell by Cell

Use the recurrence relation:

$$D[i][j] = \min \begin{cases} D[i-1][j] + 1 & \text{(Deletion)} \\ D[i][j-1] + 1 & \text{(Insertion)} \\ D[i-1][j-1] + \text{cost} & \text{(Substitution)} \end{cases}$$

Cell (1,1): comparing **c** and **c**

- Characters match → cost = 0

$$\min(2, 2, 0) = 0$$

	ε	c	u	t
ε	0	1	2	3
c	1	0		
a	2			
t	3			

Cell (1,2): comparing **c** and **u**

- Characters differ → cost = 1

$$\min(3, 1, 2) = 1$$

	ε	c	u	t
ε	0	1	2	3
c	1	0	1	
a	2			
t	3			

Cell (1,3): comparing **c** and **t**

$$\min(4, 2, 3) = 2$$

	ε	c	u	t
ε	0	1	2	3
c	1	0	1	2
a	2			
t	3			

Cell (2,1): comparing **a** and **c**

$$\min(1, 3, 2) = 1$$

	ε	c	u	t
ε	0	1	2	3
c	1	0	1	2
a	2	1		
t	3			

Cell (2,2): comparing **a** and **u**

$$\min(2, 2, 1) = 1$$

	ε	c	u	t
ε	0	1	2	3
c	1	0	1	2
a	2	1	1	
t	3			

Cell (2,3): comparing **a** and **t**

$$\min(3, 2, 2) = 2$$

	ε	c	u	t
ε	0	1	2	3
c	1	0	1	2
a	2	1	1	2
t	3			

Cell (3,1): comparing **t** and **c**

$$\min(2, 4, 3) = 2$$

Cell (3,2): comparing **t** and **u**

$$\min(2, 3, 2) = 2$$

Cell (3,3): comparing **t** and **t**

- Characters match → cost = 0

$$\min(3, 3, 1) = 1$$

Final DP Table

	ε	c	u	t
ε	0	1	2	3
c	1	0	1	2
a	2	1	1	2
t	3	2	2	1

Step 4: Final Answer

✓ Minimum Edit Distance = 1

Interpretation

Only one substitution is needed:

bash

 Copy code

cat → cut
a → u
